Full Stack Quiz App Guide React + FastAPI + SQLite + PostgreSQL

This guide walks through the complete development process of a full stack quiz application:

1. Local development setup using React, FastAPI, and SQLite
2. Subject selection and quiz history
3. Online deployment using Vercel, Render, and Supabase PostgreSQL

Stage 1 - Local Development Setup

```
Folder Structure:
quiz-app/
■■■ backend/
■■■ frontend/

Backend Stack:
- Python FastAPI
- SQLite
- SQLModel

Frontend Stack:
- React + Vite

Main Features:
- Multiple choice quiz
- Correct/wrong answer highlighting
- Randomized questions
- Subject selection
- Quiz history
- Score saving
```

Backend Setup

```
Commands:
mkdir backend
cd backend
python -m venv venv
venv\Scripts\activate
pip install fastapi uvicorn sqlmodel

Create Files:
- main.py
- database.py
- models.py
- seed.py
```

Database Architecture

```
SQLite Tables:

1. question
- id
- subject
- question_text
- option_a
- option_b
- option_c
- option_d
```

```
- correct_answer

2. quizattempt
- id
- subject
- score
- total_questions
- created_at
```

Running Backend

```
Run FastAPI server:
uvicorn main:app --reload

Backend URL:
http://127.0.0.1:8000
```

Frontend Setup

```
Commands:
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev

Frontend URL:
http://localhost:5173
```

Important Architecture Concept

```
Frontend NEVER talks directly to SQLite.

Correct Architecture:

React Frontend
↓
FastAPI Backend
↓
SQLite Database
```

Managing Questions Professionally

```
Recommended Workflow:

Excel/CSV
↓
DB Browser for SQLite
↓
SQLite Database
↓
FastAPI API
↓
React Frontend

Do NOT hardcode hundreds of questions in Python files.
```

Stage 2 - Online Deployment

```
Production Stack:
- Frontend: Vercel
- Backend: Render
- Database: Supabase PostgreSQL
Architecture:
```

```
Users
↓
Vercel Frontend
↓
Render FastAPI Backend
↓
Supabase PostgreSQL
```

Preparing Backend for PostgreSQL

```
Install packages:
pip install python-dotenv psycopg2-binary

Generate requirements:
pip freeze > requirements.txt

Environment Variable:
DATABASE_URL=postgresql://...
```

Git and GitHub Setup

```
Install Git:
https://git-scm.com/downloads

Commands:
git init
git add .
git commit -m "Initial deployment"

Create GitHub repository and push:
git remote add origin YOUR_REPO_LINK
git branch -M main
git push -u origin main
```

Deploying Backend on Render

```
Render Settings:

Root Directory:
backend

Build Command:
pip install -r requirements.txt

Start Command:
uvicorn main:app --host 0.0.0.0 --port $PORT

Environment Variable:
DATABASE_URL=YOUR_SUPABASE_URL
```

Deploying Frontend on Vercel

```
Vercel Settings:

Root Directory:
frontend

Framework:
Vite

Build Command:
npm run build

Output Directory:
dist
Environment Variable:
VITE_API_URL=YOUR_RENDER_BACKEND_URL
```

Future Upgrades

```
Recommended Next Features:

- Admin Panel
- User Authentication
- Leaderboards
- Timer System
- Difficulty Selection
- Question Categories
- Analytics Dashboard
- Multiplayer Quiz
```

Professional Development Workflow

```
Local Development
↓
GitHub Push
↓
Automatic Deployment
↓
Users Receive Updates Automatically

Whenever you add features:
1. Modify code locally
2. Push to GitHub
3. Deployment auto-updates
4. Users automatically receive updates
```